MOBILE COMPUTING PARADIGMS

Mobile computing paradigms are design principles used to build and optimize mobile systems.
They guide how mobile applications and devices should operate efficiently.
They ensure performance, reliability, and usability in mobile environments.

Mobile computing paradigms include:

- Low power computing
- Computing with limited resources
- Fault tolerance
- persistence

1. Low power computing

Low-power computing is the field dedicated to the design and manufacturing of low-power consumption circuits, programming power-aware software and applying techniques for power-efficiency.

Low power computing refers to designing mobile systems and applications that consume minimal battery power while still performing efficiently. Since mobile devices rely on batteries, energy efficiency is critical.

As digital devices shrink in size and become more portable, power consumption and energy efficiency become a critical issue. On one end, circuits designed for portable devices must target increasing battery life. On the other end, the more complex high-end circuits available at data centers have to consider power costs, cooling requirements, and reliability issues.

Why Low-Power Computing

Using lower-power computing practices suggests several rewards for individuals, business, and governments. These benefits include:

- Cutting down costs: Utility costs are a major concern in every business. Electricity prices are growing because of ever increasing power demand, and of course, natural resources are finite. While the focus on renewable energy resources could help with finite

resources, adopting strategies that lower the power consumption helps in cost reduction as well.

- Environmental Sustainability: Low-power computing is a major topic in Green IT that is concerned with minimizing the environmental impact of the IT infrastructure while maximizing efficiency. As computer systems manage their power settings more carefully, the energy generated by burning fossil fuel for powering computers will be reduced resulting in a reduced CO2 footprint.
- Extended Battery Life: In a mobile device, the battery means availability. Therefore, one of the most important attributes when purchasing a mobile device is considering its battery life. Adopting practices that lower the power consumption rate can result in a substantial increase of the battery life of a device. As a result, companies are trying to increase the battery life as much as they can to get users' blessing.
- Resource Utilization: Lowering power consumption helps in efficient and effective resource utilization which means infrastructure such as datacenters and computers. This helps in reducing the wastage of resources by matching them with the actual demand.
- Healthier life: Electronic devices draw power as long as they are operating. Being near these devices endangers humans' lives because of heat and power dissipation. Developing low-power solutions for electronics reduces heat dissipation and therefore health hazardous possibilities.

Today's mobile technologies utilize breakthroughs that squeeze more powerful performance into ever-smaller devices, boost battery life, reduce weight and heat, and virtually eliminate the need for noisy cooling fans.

System integration approaches include heterogeneous system architectures (HSAs) that combine CPUs and GPUs onto the same piece of silicon, often called accelerated processing units (APUs). This reduces typical use energy consumption by eliminating power-robbing interfaces between chips and enables on-chip management tools to efficiently allocate and reduce power between integrated components.

Devices powered by HSA help boost processing efficiency by selecting and routing workloads to the best processor for the job, enabling performance increases for both standard office and emerging “visual computing” and natural user interface applications. In short, HSA devices are designed to deliver improved performance capabilities while using less power.

Another new and promising area of power management is centered on low-power or idle states. Managing idle states is ideally suited for mobile devices as they require high performance for short periods, followed by relatively long idle intervals while the user awaits a response or observes a result. The key is to architect a design with very low-power idle states, allowing the device to enter those states as quickly as possible and creating a flexible chip configuration so there is a low-power configuration that can always be entered into quickly.

2. Computing with Limited Resources

Mobile devices operate with limited hardware resources such as RAM, CPU, storage, and bandwidth.
These limitations affect performance and application design.
Developers must create efficient and lightweight applications.

There are five key limited resources - CPU, memory, battery, data usage and time.

Mobile Cloud Computing models can be used to extend the capabilities of resource constrained mobile devices to provide the illusion of infinite, elastic resources on demand. Existing solutions for these limitations are explored, and we identify offloading computation and storage from the device as a possible solution.

Low Bandwidth and Bandwidth Variability

Wireless networks deliver lower bandwidth than wired networks, hence mobile computing designs need to be very concerned about bandwidth consumption. The deliverable bandwidth per user depends on the number of users sharing a cell. The network's capacity can be measured by its bandwidth per cubic meter. This value can be improved in two ways:

- Maintain multiple cells at different frequencies. This technique, although more flexible, is limited by the range of frequencies of the electromagnetic spectrum available for public consumption.
- Limiting transmission ranges so that more cells can fit in a given area. This preferred approach is simpler, reduces power requirements, and may decrease corruption of the signal. It is also known that transceivers covering less area can achieve higher bandwidths.

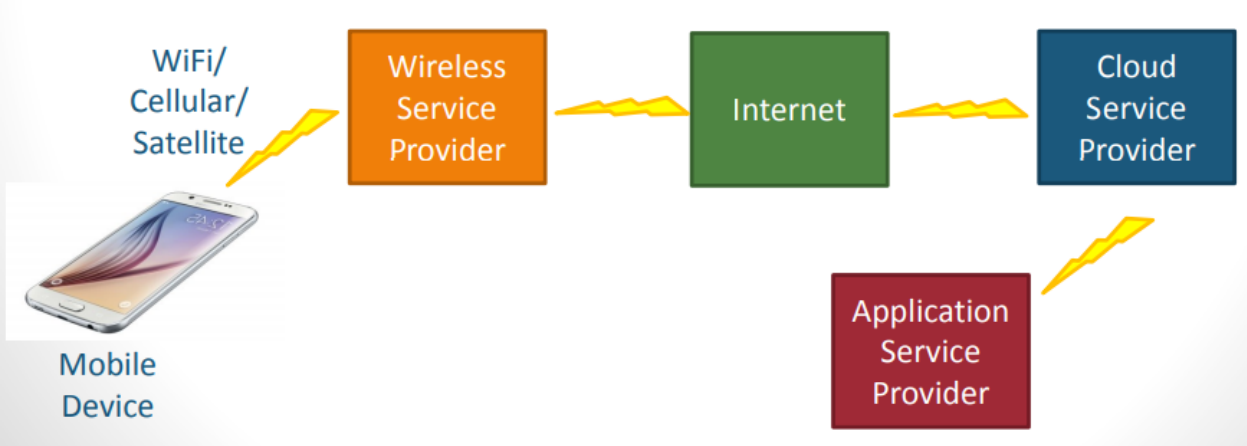
Disconnections

Since wireless communication is so susceptible to disconnection, it is of great concern when designing successful mobile computers. Resources can be allocated to handle disconnections more elegantly, or to try and prevent those disconnections from happening. In environments with frequent disconnections it is better for the mobile computer to act as a stand-alone unit rather than a mobile terminal (i.e. splitting the application and the user interface across the network).

Mobile Cloud Computing

Mobile Cloud Computing at its simplest refers to an infrastructure where both the data storage and the data processing happen outside of the mobile device. Mobile cloud applications move the computing power and data storage away from mobile phones and in to the cloud, bringing applications and mobile computing to not just smart phone users but a much broader range of mobile subscribers.

Data processing and storage are moved from the mobile device to powerful and centralized computing platforms located in clouds. They are accessed by mobile devices over a wireless connection using an app or a web browser.



3. Fault Tolerance

Fault tolerance is the ability of a system to continue working despite failures or errors. It ensures the system remains functional even when problems occur. This improves reliability in mobile computing. Mobile agent is a program that can migrate autonomously from one environment to another. Many factors affect execution of mobile agents during its life cycle. Errors may occur on the server, or during communication. Error probability further increases with longer path. In mobile agent computing environment any component of the network - node, link, or agent may fail at any time, thus preventing them from continuing their execution. Therefore, fault-tolerance is a vital issue for the deployment of mobile agent systems.

Fault Management

Fault tolerance is the ability of a system to perform its function correctly even in the presence of internal faults. The purpose of fault tolerance is to increase the dependability of

a system. A complimentary but separate approach to increasing dependability is fault prevention. This consists of techniques, such as inspection, whose intent is to eliminate the circumstances by which faults arise.

Faults can be classified as transient or permanent. A transient fault will eventually disappear without any apparent intervention, whereas a permanent one will remain unless it is removed by some external agency. While it may seem that permanent faults are more severe, from an engineering perspective they are much easier to diagnose and handle. The intermittent transient faults that recur often unpredictably are the most problematic.

The mobile computing environment is characterized by limited resource availability, high mobility, low bandwidth and frequent disconnection. Because of high mobility, network connectivity is often lost or degraded. Users may leave the area of network connectivity, or may enter into an area of higher interference.

Mobile devices need to be lightweight and small in order to facilitate high mobility. This leads to the use of small batteries resulting in low available power. Processing of mobile computing applications is also halted due to such battery drain. Again, frequent disconnections can halt the applications.

All such features make the mobile computing environment more susceptible to transient faults.

Faults are an integral part of the world of wireless technology due to the aforementioned problems of frequent disconnections, narrow bandwidth, lack of resources and high mobility. Hence, the necessity of providing better fault-tolerant mechanisms for ensuring reliable working of mobile devices cannot be ignored.

Mobile computing requires an integrated solution towards fault-tolerant mechanisms, for example battery power cannot be saved if some check-pointing algorithm uses many system messages to complete. It is required to provide a complete fault-tolerance solution for various applications of different natures.

Availability is defined here in terms of providing fault-tolerance to running applications and enhancing resources for future mobile computing applications.

The following steps show how to manage various types of faults in a mobile computing environment in order to get better availability for mobile computing applications on mobile devices. Various error recovery routines e.g., Error_Routine_FEC, Error_Routine_Chp etc., can be designed and invoked depending on the nature of various faults detected in such mobile computing environment for attaining fault tolerance.

***

If fault == **Packet Loss**, Then: Display "Nature of Fault is Environment" Display "Presence of Fault is Transient" Display "Suggested Solutions is Forward Error Correction" Run Error_Routine_FEC

Else If fault == **Mobile Crash**, Then: Display "Nature of Fault is Software" Display "Presence of Fault is Transient" Display "Suggested Solutions is Checkpointing". Run Error_Routine_Chp

Else If fault == **Transaction Failure**, Then: Display "Nature of Fault is Software" Display "Presence of Fault is Transient" Display "Suggested Solutions is Checkpointing". Run Error_Routine_Chp

Else If fault == **Out of Reach**, Then: Display "Nature of Fault is Environment" Display "Presence of Fault is Transient" Display "Suggested Solutions is Hoarding". Run Error_Routine_Hrd

Else If fault == **Bandwidth Reduction**, Then: Display "Nature of Fault is Environment" Display "Presence of Fault is Transient" Display "Suggested Solutions is Prefetching , prenegotiation". Run Error_Routine_PP

Else If fault == **Memory Capacity**, Then: Display "Nature of Fault is Architectural" Display "Presence of Fault is Permanent" Display "Suggested Solutions is Prefetching , Buffer sharing". Run Error_Routine_PBShr

Else If fault ==**Problem in Battery**, Then: Display "Nature of Fault is Architectural" Display "Presence of Fault is Permanent" Display "Suggested Solutions is Strategies for Power Saving". Run Error_Routine_SPWRSV

End If

Persistence

Persistence is the ability of a system to store data permanently so that it is not lost when an application is closed or a device is restarted. This means that once data is saved, it can still be accessed later even after interruptions such as power loss or app shutdown. It ensures that important information is retained over time and can be reused whenever needed, supporting long-term data availability in mobile systems.

Persistence is important because it prevents the loss of user data, ensuring that information such as messages, files, or settings is safely stored. It allows users to continue working or interacting with an application from the exact point where they left off. This continuity improves convenience and reliability, which greatly enhances the overall user experience in mobile applications.

Persistence is achieved through different types of storage systems depending on the application needs. Local storage saves data directly on the mobile device, making it accessible even without internet connection. Databases are used to store structured data in an organized way for applications. Cloud storage keeps data online, allowing users to access it from different devices anytime through the internet.